

PLANTILLA DE REPORTE DE ERROR

Número de Bug	<i>BC001</i>
Nombre del Tester	<i>Jhojan Erick Romero Morales</i>
Título	<i>Vulnerabilidad lógica permite transferencias con montos matemáticamente negativos alterando los saldos</i>
Día del Reporte	<i>20/09/2026</i>
Sistema Operativo	<i>Windows 11</i>
Browser	<i>Back-end API (Postman / Ejecución en IntelliJ)</i>
Prioridad	<i>Alta / Crítica</i>
Asignado a:	<i>Desarrollador Back-end / Módulo de Servicios (CuentaServiceImpl)</i>

Descripción

HISTORIA: TRANSFERENCIA EXITOSA (HU-01) y DÉBITO EN CUENTA (HU-05)

CRITERIO AFECTADO: REQ-05 ("El sistema debe poder aplicar un débito directo previniendo montos ilógicos") y el CA de HU-01 ("El sistema procesa la transferencia restando del origen y sumando al destino"). El método transferir en la clase CuentaServiceImpl no implementa una validación para verificar que el parámetro monto sea estrictamente mayor a cero antes de procesar la transacción. Al inyectar un valor numérico negativo, el método interno debito realiza una resta matemática de un número negativo (ej. saldo - (-100)), lo cual invierte la operación, sumando dinero a la cuenta de origen de forma irregular e impactando la integridad financiera.

Pasos para la reproducción

1. Iniciar el entorno de pruebas unitarias en IntelliJ para la clase CuentaServiceImplTest. Hacer
2. Diseñar un caso de prueba que invoque el método service.transferir.
3. Enviar los parámetros con un monto en negativo.
4. Consultar los saldos posteriores a la transacción mediante aserciones.
5. Datos de prueba utilizados: origen: 1 (Saldo inicial: 1000) | destino: 2 (Saldo inicial: 2000) | monto: -100 | bancold: 1

Resultado Esperado

El sistema debe interceptar el valor ilógico antes de la transacción, abortar la operación sin invocar a la base de datos y lanzar una excepción controlada (ej. IllegalArgumentException) indicando que no se admiten montos negativos.

Resultado Actual

El sistema procesa exitosamente la solicitud como válida. La cuenta de origen termina con un saldo inflado de 1100 (creación de dinero irreal) y la cuenta de destino sufre un descuento injustificado bajando a 1900, vulnerando las reglas de negocio.

Impresión de pantalla / video y otros

```
✓ CuentaServiceImplTest 3 sec 728 ms ✗ 1 test failed 1 test total, 3 sec 728 ms
  ✗ testTransferirMontoNeg: 875 ms
    "D:\Apache\NetBeans\jdk\bin\java.exe" ...
    sept 21, 2026 12:17:18 A. M. org.junit.platform.launcher.core.EngineDiscoveryOrchestrator Lambda$logTestDescriptorExclusionReasons$7
    INFORMACIÓN: 0 containers and 1 tests were testDebitoModelo() doesn't match
    sept 21, 2026 12:17:18 A. M. org.junit.platform.launcher.core.EngineDiscoveryOrchestrator Lambda$logTestDescriptorExclusionReasons$7
    INFORMACIÓN: 0 containers and 1 tests were testTransferirPositivo() doesn't match
    sept 21, 2026 12:17:18 A. M. org.junit.platform.launcher.core.EngineDiscoveryOrchestrator Lambda$logTestDescriptorExclusionReasons$7
    INFORMACIÓN: 0 containers and 1 tests were testTransferirFondosInsuficientes() doesn't match

    org.opentest4j.AssertionFailedError: El sistema debería bloquear transferencias con montos matemáticamente negativos ==> Expected
    java.lang.IllegalArgumentException to be thrown, but nothing was thrown.
    > <3 internal lines>
    > at test.springboot.app.services.CuentaServiceImplTest.testTransferirMontoNegativoLanzaExcepcion(CuentaServiceImplTest.java:79)
    > <4 internal lines>
    > <62 folded frames>

    |
    Process finished with exit code -1
```